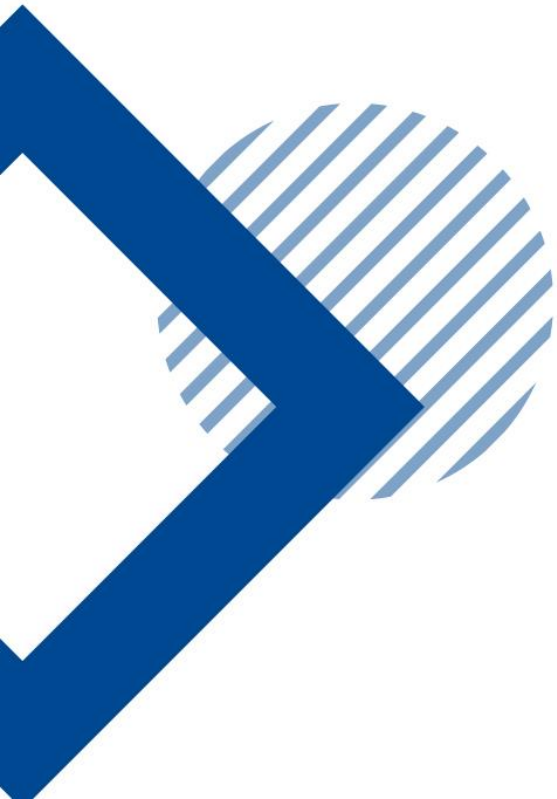
Введение в интернет-программирование

Интернет-программирование





2022

Клиент-серверный подход – 1

Сервер

Аппаратный Сервер - выделенный компьютер для выполнения определенных функций:

- Работа без участия человека
- Надежность
- Аппаратный контроль
- Производительность
- Специальное внешнее исполнение

Программный Сервер - программное обеспечение, обслуживающее запросы от клиентов:

- Разные типы серверов (web, file, printer, terminal...)
- Вычислительная работа
- Обслуживание запросов клиентов
- Предоставляет доступ к ресурсам
- Параллельное обслуживание
- Формат взаимодействия - протокол

Клиент-серверный подход – 2

Клиент

- Программа, посылающая запросы на сервер
- Взаимодействует по протоколу
- Типы клиентов - браузер, сетевая игра
- Терминал, сервис печати, сетевой диск и т. д.

Взаимодействие сервера и клиента

1. Клиент: отправка запроса
2. Сервер:
- получение запроса
- обработка запроса + подготовка ответа
- отправка ответа

3. Клиент: получение ответа

Web-сервер и Web-браузер

Web Сервер

- Apache
- Nginx
- IIS
- Встроенные серверы
- Серверы приложений (https://youtu.be/BcmUOmvl1N8)

Web Браузер

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Opera, Vivaldi
- Safari

Web Браузер

Процесс работы:

- Отправка запроса
- Загрузка заголовков ответа
- Загрузка HTML кода страницы
- Построение DOM
- Загрузка связанных ресурсов
- Отображение
- Исполнение JavaScript

Формирование HTML-страницы

Разработка и верстка:

- HTML
- CSS
- JavaScript

Наполнение:

- Медиафайлы
- Содержимое

Программирование Web Сервера

Разработка:

- Разработка приложения (код)
- Моделирование базы данных (структура)
- Формирование логики работы с БД (SQL)
- Разработка API (Rest, XML, JSON)
- Разработка шаблонов HTML-страниц (HTML, CSS, JavaScript)

Развертывание и настройка:

- Linux (оболочка, эмулятор терминала, структура ФС, основные команды)
- Автоматизация (bash, python, php)

HTTP – 1

Основные понятия:

- Протокол для передачи данных между сервером и клиентом
- Используется в Web для передачи информации/страниц с сайтов
- URL – запрос логического документа с сервера
- Не сохраняет состояние между запросами
- Заголовки

Структура протокола:

- Стартовая строка
- Заголовки
- Тело сообщения

HTTP – 2

Пример запроса:

GET /wiki/страница HTTP/1.1
Host: ru.wikipedia.org
User-Agent: Mozilla/5.0 (X11; U; Linux i686; ru; rv:1.9b5) Gecko/2008050509 Firefox/3.0b5
Accept: text/html
Connection: close

Пример ответа:

HTTP/1.1 200 OK
Date: Wed, 11 Feb 2009 11:20:59 GMT
Last-Modified: Wed, 11 Feb 2009 11:20:59 GMT
Content-Language: ru
Content-Type: text/html; charset=utf-8
Content-Length: 1234
Connection: close
(пустая строка)
(далее следует запрошенная страница в HTML)

HTTP – 3

Основные коды ответа:

- 200 OK
- 404 Not Found
- 403 Forbidden
- 500 Internal Server Error

Тип содержимого:

- Content-Type: text/html
- Content-Type: application/msword
- Content-Type: image/jpg
- Content-Type: application/zip

Методы HTTP – 1

Самое первое слово в запросе – это метод. Указывается заглавными буквами.
GET /wiki/страница HTTP/1.1
Host: ru.wikipedia.org
Метод указывает на операцию над ресурсом.

- OPTIONS
- HEAD
- PUT
- PATCH
- DELETE

Методы HTTP – 2

Метод GET:

- Используется для запроса содержимого указанного ресурса
- Согласно стандарту HTTP, запросы типа GET считаются идемпотентными (повторное действие над объектом не изменяет его)

Метод POST:

- Применяется для передачи пользовательских данных заданному ресурсу
- В отличие от метода GET, метод POST не считается идемпотентным, то есть многократное повторение одних и тех же запросов POST может возвращать разные результаты

Инструменты и технологии создания web-приложений

- Операционная система: Linux
- IDE: IntelliJ IDEA, Eclipse, NetBeans, PHPStorm, Plain Text Editors, Vim
- Version Control System (VCS):
  - История изменений
  - Автор
  - Хронология
  - Change set
  - Branches (ветки)

Фреймворк Spring – 1

Почему Spring?

Основные преимущества:

- Скорость разработки
- Простота разработки

Основной недостаток – необходимо обладать знаниями и навыками в:

- CDI
- AOP
- JPA
- JDBC
- JPQL
- Hibernate
- MVC
- REST
- ActiveMQ
- OPS и т. д.

Фреймворк Spring – 2

Какие приложения можно разрабатывать на Spring?

1. Микросервисы
2. Реактивные приложения
3. Приложения для облачных платформ
4. Веб-приложения
5. Бессерверные приложения
6. Приложения, управляемые событиями
7. Системы пакетной обработки

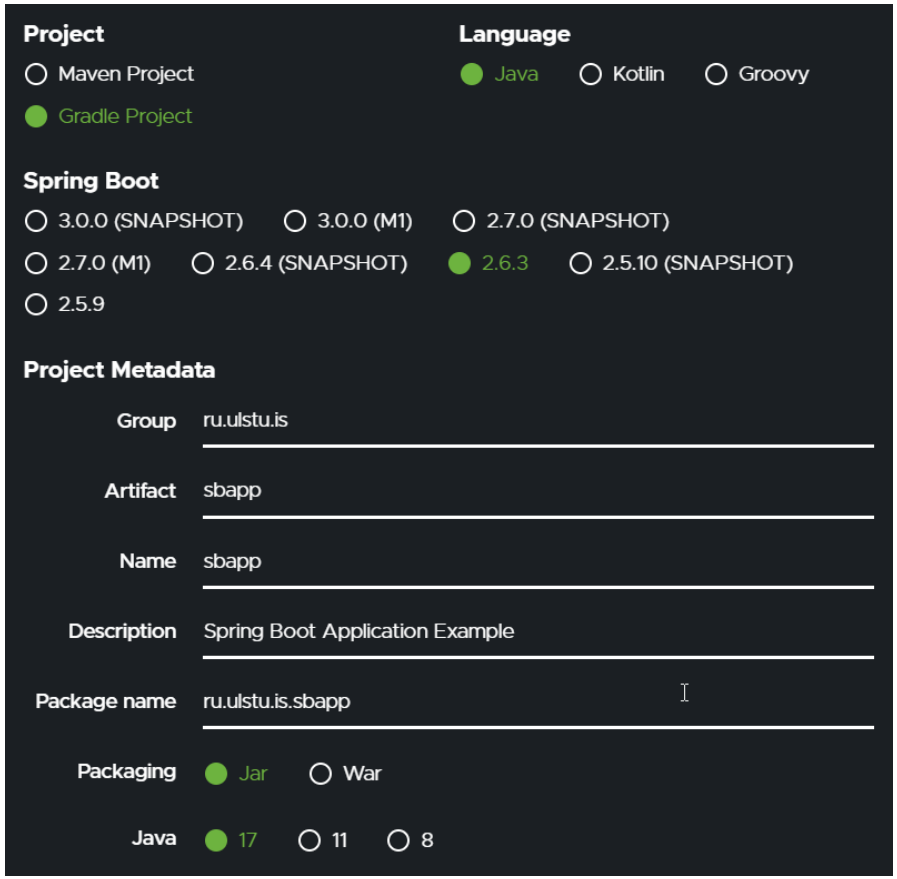
Фреймворк Spring Boot

Spring Boot – фреймворк для работы с фреймворком Spring:

1. Используется для облегчения разработки приложений на основе фреймворка Spring
2. Объединяет компоненты фреймворка Spring и различные сторонние библиотеки
3. Предоставляет начальные настройки для компонентов фреймворка Spring и сторонних библиотек, подходящие для большинства приложений
4. Позволяет создавать самодостаточные приложения, не требующие наличия сервера приложений

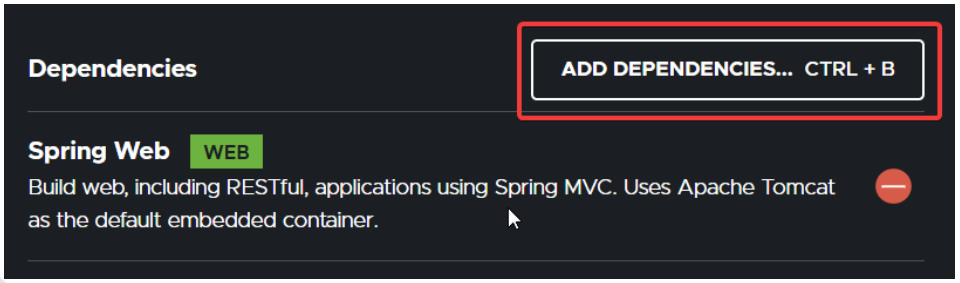
Генерация минимального Spring Boot приложения – 1

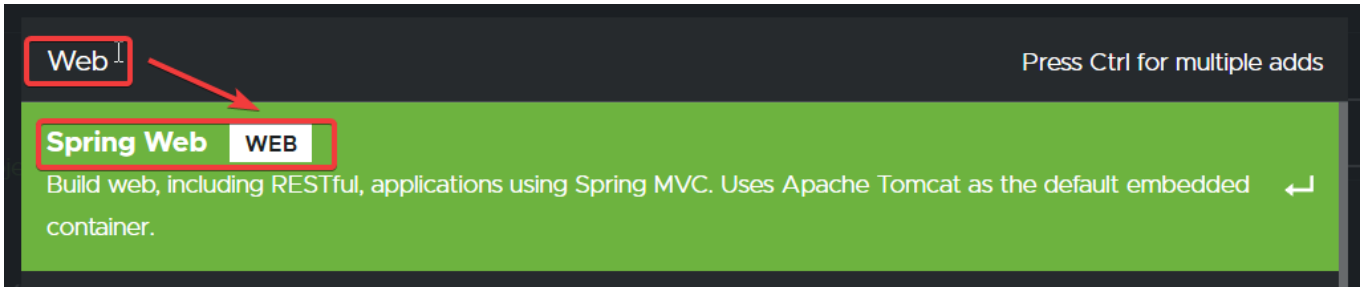
https://start.spring.io/ (дата обращения: 10.02.2022)

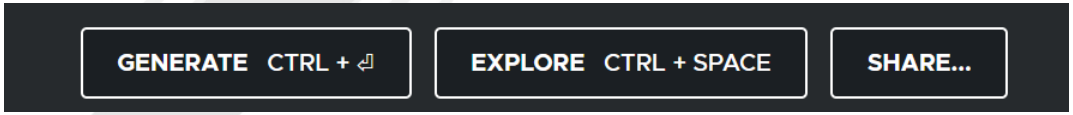


Генерация минимального Spring Boot приложения – 2

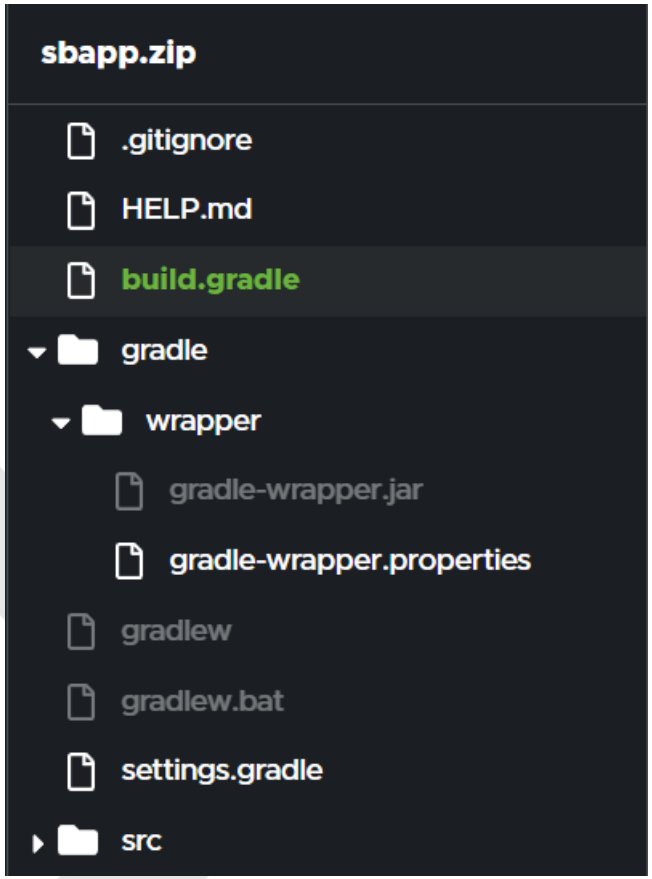
https://start.spring.io/ (дата обращения: 10.02.2022)







Генерация минимального Spring Boot приложения – 3




```
plugins {
 id 'org.springframework.boot' version '2.6.3'
 id 'io.spring.dependency-management' version '1.0.11.RELEASE'
 id 'java'
}

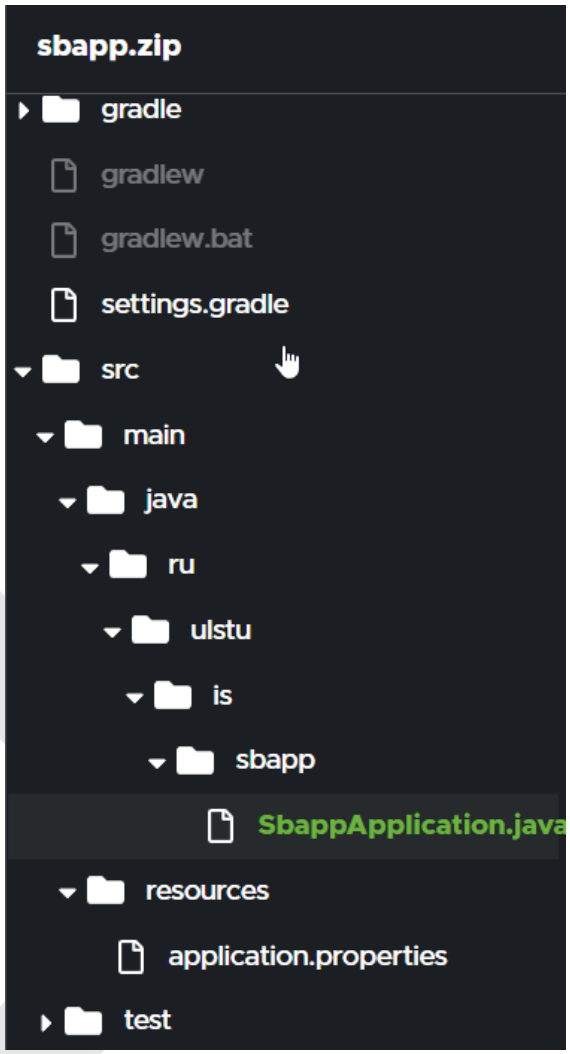
group = 'ru.ulstu.is'
version = '0.0.1-SNAPSHOT'
sourceCompatibility = '17'

repositories {
 mavenCentral()
}

dependencies {
 implementation 'org.springframework.boot:spring-boot-starter-web'
 testImplementation 'org.springframework.boot:spring-boot-starter-test'
}

tasks.named('test') {
 useJUnitPlatform()
}
```

Генерация минимального Spring Boot приложения – 4




```
package ru.ulstu.is.sbapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SbappApplication {

 public static void main(String[] args) {
  SpringApplication.run(SbappApplication.class, args);
 }

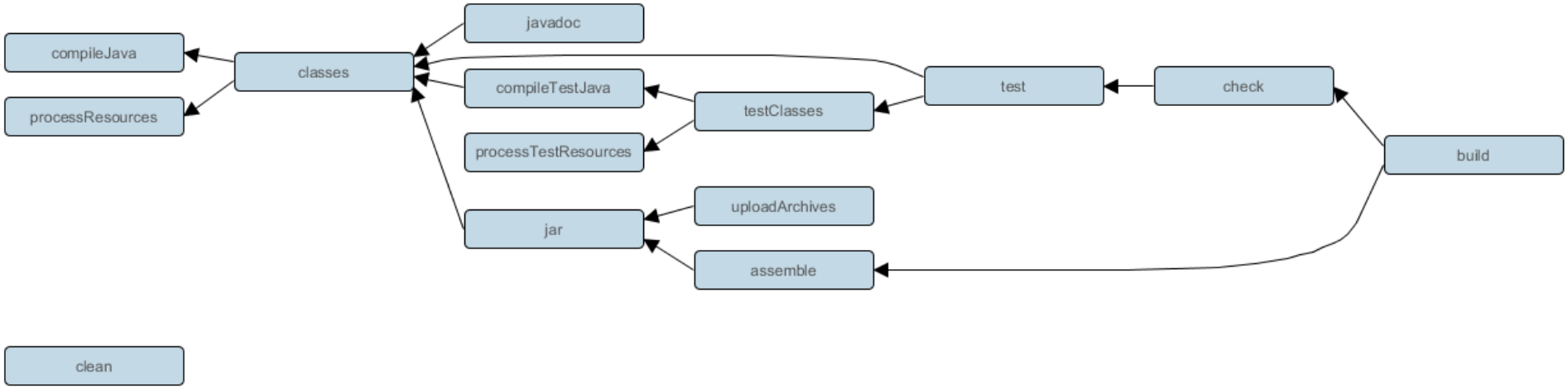
}
```

Запуск минимального Spring Boot приложения

Основные задачи Gradle:

- tasks
- clean
- build
- bootRun
- check

./gradlew bootRun – запуск
./gradlew clean build – сборка

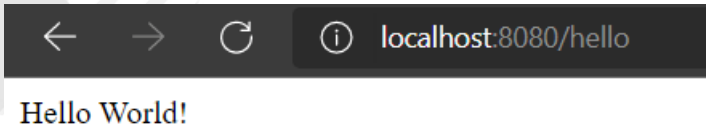


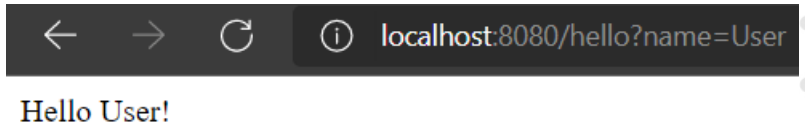
Пример простого web-приложения на Spring Boot

```
@SpringBootApplication
@RestController
public class SbappApplication {

    public static void main(String[] args) {
        SpringApplication.run(SbappApplication.class, args);
    }

    @GetMapping("/hello")
    public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
        return String.format("Hello %s!", name);
    }
}
```





Спасибо за внимание!

Ссылки:

1. https://spring.io/
2. https://spring.io/projects/spring-boot
3. https://start.spring.io/
4. https://spring.io/quickstart



2022